

Operativni sistemi

Datotečni sistem

(Laboratorijska vežba III-3)

Zadatak 1

Korišćenjem programskog jezika C napisati UNIX/Linux program koji u zadanom direktorijumu lista nazive svih datoteka koje u svom nazivu sadrže zadati string. Putanja do direktorijuma i string koj se traži zadaju se kao argument komandne linije.

```
#include <stdlib.h>
#include <stdio.h>
#include <string.h>
#include <sys/types.h>
#include <dirent.h>

int main(int argc, char* argv[])
{
    DIR* dp;
    struct dirent* dirp;
    if (argc != 3)
    {
        printf("Neophodna su dva argumenta (putanja do direktorijuma i string koji se traži)\n");
        exit(-1);
    }

    if ((dp = opendir(argv[1])) == NULL)
    {
        printf("Greska prilikom otvaranja direktorijuma %s!\n", argv[1]);
        exit(-1);
    }

    while( (dirp = readdir(dp)) != NULL )
    {
        if (strstr(dirp->d_name, argv[2]) != NULL)
            printf("%s\n", dirp->d_name);
    }

    closedir(dp);
}
```

Zadatak 2

Korišćenjem programskog jezika C napisati UNIX/Linux program koji prikazuje statistiku za zadati direktorijum (broj poddirektorijuma, regularnih datoteka i linkova). Putanja do direktorijuma (apsolutna ili relativna) se zadaje kao argument komandne linije.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/types.h>
#include <dirent.h>
#include <sys/stat.h>

int main(int argc, char* argv[])
```

```

{
    DIR* dp;
    struct dirent* dirp;
    int brfldr, brdat, brlnk;
    struct stat statbuf;
    int result;
    char path[255];

    if (argc != 2)
    {
        printf("Neophodan je jedan argument (putanja do direktorijuma)!\n");
        exit(1);
    }

    if ((dp = opendir(argv[1])) == NULL)
    {
        printf("Greska prilikom otvaranja foldera %s!\n", argv[1]);
        exit(1);
    }

    brfldr = brdat = brlnk = 0;

    while( (dirp = readdir(dp)) != NULL )
    {
        //formiranje putanje do datoteke
        strcpy(path, argv[1]);
        strcat(path, "/");
        strcat(path, dirp->d_name);

        printf("Test: %s\n", path);

        if ((result = stat(path, &statbuf)) == -1)
        {
            printf("Neuspesno citanje podataka o objektu %s\n:", dirp->d_name);
            continue;
        }

        if (S_ISDIR(statbuf.st_mode))
        {
            printf("Objekat: %s je direktorijum\n", dirp->d_name);
            brfldr++;
        }

        if(S_ISREG(statbuf.st_mode))
        {
            printf("Objekat: %s je regularna datoteka\n", dirp->d_name);
            brdat++;
        }

        if(S_ISLNK(statbuf.st_mode))
        {
            printf("Objekat: %s je link\n", dirp->d_name);
            brlnk++;
        }
    }

    printf("Statistika za folder: %s\nPodfoldera: %d\nReg. datoteka: %d\nLinkova: %d\n", argv[1], brfldr, brdat, brlnk);
}

```

```
    closedir(dp);  
}
```

Zadatak 3

Korišćenjem programskog jezika C napisati Linux/UNIX program koji u zadatom direktorijumu (naziv direktorijuma se zadaje kao prvi argument komandne linije) pronalazi i štampa nazive svih datoteka koje u imenu sadrže string koji se zadaje kao drugi argument komandne linije. Pretraživanje se obavlja rekurzivno u zadatom direktorijumu i svim njegovim poddirektorijumima.

```
#include <stdio.h>  
#include <stdlib.h>  
#include <string.h>  
#include <sys/types.h>  
#include <dirent.h>  
#include <sys/stat.h>  
  
void processdir(char* foldername, char* str){  
    DIR* dp;  
    struct dirent* dirp;  
    int brfldr, brdat, brlnk;  
    struct stat statbuf;  
    int result;  
  
    if ((dp = opendir(foldername)) == NULL)  
    {  
        printf("Greska prilikom otvaranja foldera %s!\n", foldername);  
        return;  
    }  
  
    while( (dirp = readdir(dp)) != NULL )  
    {  
        char tmp[1024] = "";  
        strcat(tmp, foldername);  
        strcat(tmp, "/");  
        strcat(tmp, dirp->d_name);  
  
        printf("Obradjujem %s u %s.\n", dirp->d_name, tmp);  
        if ((result = stat(tmp, &statbuf)) == -1)  
        {  
            printf("Neuspesno citanje podataka o objektu: %s\n", tmp);  
            continue;  
        }  
  
        if (strstr(dirp->d_name, str) != NULL)  
            printf("Pronasao %s\n", tmp);  
  
        if (S_ISDIR(statbuf.st_mode)  
            && strcmp(dirp->d_name, ".") != 0  
            && strcmp(dirp->d_name, "..") != 0)  
        {  
            printf("Otvaram folder %s.\n", tmp);  
            processdir(tmp, str);  
        }  
    }  
}  
  
closedir(dp);
```

```

}

int main(int argc, char* argv[])
{
    if (argc != 3)
    {
        printf("Neophodna su dva argumenta (putanja do foldera i string koji se traži)\n");
        exit(1);
    }

    processdir(argv[1], argv[2]);
}

```

Zadatak 4

Korišćenjem programskog jezika C napisati Linux/UNIX program koji predstavlja verziju **ls** komande. Program kao argument ima ime direktorijuma, a prikazuje listu direktorijuma u njemu, liniju po liniju, sa sledećim podacima: ime direktorijuma, broj datoteka koje se u njemu nalaze, veličinu u bajtovima (veličina predstavlja zbir veličina datoteka koje se nalaze u direktorijumu).

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/types.h>
#include <dirent.h>
#include <sys/stat.h>

#define MAX_NAME 1024

void processdir(char * path, int * count, int * size);

int main(int argc, char * argv[])
{
    char dirname[MAX_NAME];
    char path[MAX_NAME];
    struct stat statbuf;
    DIR * dp;
    struct dirent * dirp;
    int count, size;

    if (argc < 2)
    {
        printf("Nema dovoljno ulaznih argumenata\n");
        exit(-1);
    }

    strcpy(dirname, argv[1]);

    //proveravamo da li je uneta putanja validna
    if (stat(dirname, &statbuf) < 0)
    {
        printf("Dosló je do greske prilikom ocitavanja status zadate putanje\n");
        exit(-1);
    }

    //da li je uneta putanja do direktorijuma
    if (!S_ISDIR(statbuf.st_mode))

```

```

{
    printf("Uneti argument ne predstavlja putanju do direktorijuma\n");
    exit(-1);
}

dp = opendir(dirname);
if (dp == NULL)
{
    printf("Dosló je do greske prilikom otvaranja direktorijuma\n");
    exit(-1);
}

while((dirp = readdir(dp)) != NULL)
{
    if (strcmp(dirp->d_name, ".") == 0
        || strcmp(dirp->d_name, "..") == 0)
        continue;

    strcpy(path, dirname);
    strcat(path, "/");
    strcat(path, dirp->d_name);

    stat(path, &statbuf);
    if (S_ISDIR(statbuf.st_mode))
    {
        processdir(path, &count, &size);
        printf("%s\t%d\t%d\n", dirp->d_name, count, size);
    }
}

closedir(dp);

return 0;
}

void processdir(char * path, int * count, int * size)
{
    DIR * dp;
    struct dirent * dirp;
    char path1[MAX_NAME];
    struct stat statbuf;

    dp = opendir(path);

    if (dp == NULL)
    {
        printf("Greska prilikom otvaranja direktorijuma\n");
        exit(-1);
    }

    *count = 0;
    *size = 0;

    while ((dirp = readdir(dp)) != NULL)
    {
        strcpy(path1, path);
        strcat(path1, "/");
        strcat(path1, dirp->d_name);
    }
}

```

```

    stat(path1, &statbuf);
    if (S_ISREG(statbuf.st_mode))
    {
        *count = *count + 1;
        *size = *size + statbuf.st_size;
    }
}

closedir(dp);
}

```

Zadatak 5

Korišćenjem programskog jezika C napisati Linux/UNIX program koji u zadanom direktorijumu rekurzivno (maksimalno do dubine 3) briše sve regularne datoteke čija je veličina veća od 10KB. Za brisanje datoteka koristiti **rm** komandu Linux operativnog sistema.

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <dirent.h>
#include <sys/stat.h>

#define MAX_NAME 1024
void processdir(char * path, int level);

int main(int argc, char * argv[])
{
    char dirname[MAX_NAME];
    char path[MAX_NAME];
    struct stat statbuf;
    DIR * dp;
    struct dirent * dirp;
    int count, size;

    if (argc < 2)
    {
        printf("Nema dovoljno ulaznih argumenata\n");
        exit(-1);
    }
    strcpy(dirname, argv[1]);

    //proveravamo da li je uneta putanja validna
    if (stat(dirname, &statbuf) < 0)
    {
        printf("Dosló je do greske prilikom ocitavanja statusa unete putanje\n");
        exit(-1);
    }

    //da li je uneta putanja do direktorijuma
    if (!S_ISDIR(statbuf.st_mode))
    {
        printf("Uneti argument ne predstavlja putanu do direktorijuma\n");
        exit(-1);
    }
}

```

```

    processdir(dirname, 1);

    return 0;
}
void processdir(char * path, int level)
{
    DIR * dp;
    struct dirent * dirp;
    char path1[MAX_NAME];
    struct stat statbuf;

    dp = opendir(path);
    if (dp == NULL)
    {
        printf("Greska prilikom otvaranja direktorijuma\n");
        exit(-1);
    }

    while ((dirp = readdir(dp)) != NULL)
    {
        if (strcmp(dirp->d_name, ".") == 0
            || strcmp(dirp->d_name, "..") == 0)
            continue;

        strcpy(path1, path);
        strcat(path1, "/");
        strcat(path1, dirp->d_name);

        stat(path1, &statbuf);
        if (S_ISREG(statbuf.st_mode) && statbuf.st_size > 10 * 1024)
        {
            if (fork() == 0)
            {
                printf("Delete file: %s\n", path1);
                execlp("rm", "rm", path1, NULL);
            }
            else
            {
                wait(NULL);
            }
        }
        else if (S_ISDIR(statbuf.st_mode) && level < 3)
        {
            processdir(path1, level + 1);
        }
    }

    closedir(dp);
}

```

Zadatak 6

Korišćenjem programskog jezika C napisati Linux/UNIX program koji u zadatom direktorijumu određuje naziv najveće datoteke. Ime direktorijuma se programu prosleđuje kao argument komandne linije.

```

#include <stdio.h>
#include <stdlib.h>

```

```

#include <string.h>
#include <sys/types.h>
#include <dirent.h>
#include <sys/stat.h>

#define MAX_NAME 1024

int main(int argc, char * argv[])
{
    char dirname[MAX_NAME];
    char path[MAX_NAME];
    char maxpath[MAX_NAME];
    int size;
    struct stat statbuf;
    DIR * dp;
    struct dirent * dirp;

    if (argc < 2)
    {
        printf("Nema dovoljno ulaznih argumenata\n");
        exit(-1);
    }

    strcpy(dirname, argv[1]);
    //proveravamo da li je uneta putanja validna
    if (stat(dirname, &statbuf) < 0)
    {
        printf("Doslo je do greske prilikom ocitavanja statusa unete putanje\n");
        exit(-1);
    }

    //da li je uneta putanja do direktorijuma
    if (!S_ISDIR(statbuf.st_mode))
    {
        printf("Uneti argument ne predstavlja putanju do direktorijuma\n");
        exit(-1);
    }

    dp = opendir(dirname);
    if (dp == NULL)
    {
        printf("Doslo je do greske prilikom otvaranja direktorijuma\n");
        exit(-1);
    }

    size = 0;
    while((dirp = readdir(dp)) != NULL)
    {
        strcpy(path, dirname);
        strcat(path, "/");
        strcat(path, dirp->d_name);

        stat(path, &statbuf);
        if (S_ISREG(statbuf.st_mode))
        {
            if (statbuf.st_size > size)
            {
                strcpy(maxpath, path);
                size = statbuf.st_size;
            }
        }
    }
}

```

```
    }  
  
    }  
}  
  
printf("Datoteka %s ima maksimalnu velicinu %dB\n", maxpath, size);  
closedir(dp);  
  
return 0;  
}
```